Traitement par Lot et Streaming

avec Apache Spark

Module	Big Data
Outil principal	Apache Spark 3.5.0 sur Hadoop 3.3.6
Environnement	Docker — hadoop-master / worker1 / worker2
Langage	Scala (Spark-Shell) — Java (Maven)
Auteur	BOUGOUM Cheick Abdoul Aziz
Date	18 mars 2026
Classe	INGC2-ESMT
Enseignant	Dr Mohamed Kaba KEITA

Rapport rédigé dans le cadre du TP2

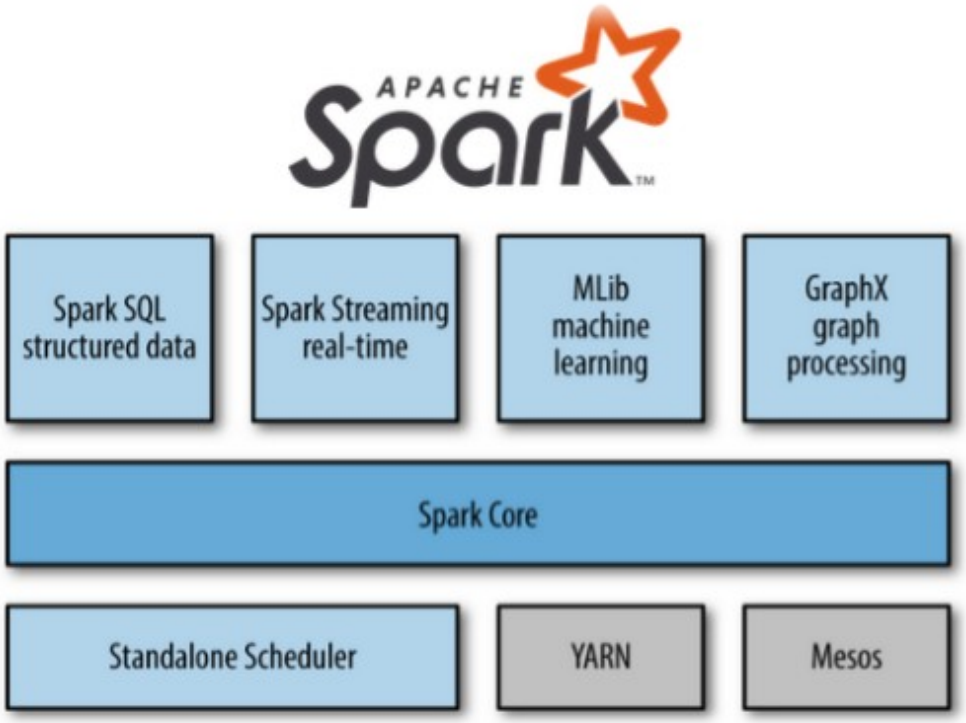
TABLE DES MATIÈRES

1. Introduction....3
1.1 Objectifs du TP....3
1.2 Environnement technique....4
2. Présentation d'Apache Spark....4
2.1 Architecture de Spark....4
2.2 Modules principaux....5
2.3 Intégration avec Hadoop YARN....5
3. Installation et démarrage du cluster....5
3.1 Lancement des contenaires Docker....5
3.2 Vérification des démons....6
4. Test de Spark avec Spark-Shell (Scala)....6
4.1 Préparation du fichier de test....6
4.2 Lancement de Spark-Shell....6
4.3 WordCount en Scala....7
4.4 Explication des transformations et action....7
4.5 Résultats obtenus....8
4.6 Exemple : relever les mots les plus fréquents dans un fichier....8
4.6.1 Explication des transformations et action....9
5. Spark Batch en Java (WordCount)....10
5.1 Création du projet Maven....10
5.2 Dépendances pom.xml....10
5.3 Code WordCountTask.java....10
5.4 Test en local....11
5.5 Déploiement sur le cluster....12
5.6 Modes de déploiement YARN....13
5.7 Résultats....13
5.7.1 Résultat en mode YARN cluster....13
6. Spark Streaming (Structured Streaming)....15
6.1 Présentation....15
6.2 Configuration du projet Maven....15
6.3 Code de la classe Stream.java....15
6.4 Déploiement et test sur le cluster....15
6.5 Résultats....16
7. Conclusion....18

1. INTRODUCTION

Ce rapport présente le compte-rendu du TP2 consacré au traitement par lot (batch) et au traitement en streaming à l'aide d'Apache Spark. L'objectif est de mettre en pratique les concepts fondamentaux de Spark RDDs, transformations, actions et Structured Streaming dans un environnement distribué basé sur Docker et Hadoop YARN.

L'énoncé de ce TP est adapté du TP2 « Traitement par Lot et Streaming avec Spark » de Lilia Sfaxi (INSAT Tunisie), disponible à l'adresse https://insatunisia.github.io/TP-BigData/tp2/. Ce rapport documente l'exécution du TP sur l'environnement de l'ESMT.



Pile logicielle d'Apache Spark. Source : Lilia Sfaxi, INSAT Tunisie, « TP Big Data », TP2 (https://insatunisia.github.io/TP-BigData/tp2/).

1.1 OBJECTIFS DU TP

- Comprendre l'architecture d'Apache Spark et son intégration avec Hadoop YARN
- Manipuler les RDDs via Spark-Shell en Scala
- Développer et déployer un job Spark Batch en Java (WordCount)
- Implémenter un traitement en continu avec Spark Structured Streaming
- Exécuter les jobs en mode local puis en mode cluster YARN

1.2 ENVIRONNEMENT TECHNIQUE

Apache Hadoop	3.3.6
Apache Spark	3.5.0
Docker	latest
Java	1.8
IDE	Visual Studio Code 1.85.1
OS	Ubuntu 24.04 LTS

2. PRÉSENTATION D'APACHE SPARK

Apache Spark est un moteur de traitement de données distribué, rapide et polyvalent. Il fournit des APIs de haut niveau en Java, Scala, Python et R, et supporte plusieurs paradigmes de traitement grâce à ses modules spécialisés.

2.1 ARCHITECTURE DE SPARK

Spark repose sur une abstraction centrale appelée RDD (Resilient Distributed Dataset), qui représente une collection d'éléments partitionnés à travers les nœuds du cluster. Les RDDs supportent deux types d'opérations fondamentales :

- Transformations (lazy) : map, flatMap, filter, reduceByKey, groupByKey... Ces opérations créent un nouveau RDD à partir d'un existant sans déclencher de calcul immédiat.
- Actions : count, collect, saveAsTextFile, take... Ces opérations déclenchent l'exécution réelle du graphe de transformations et retournent un résultat.

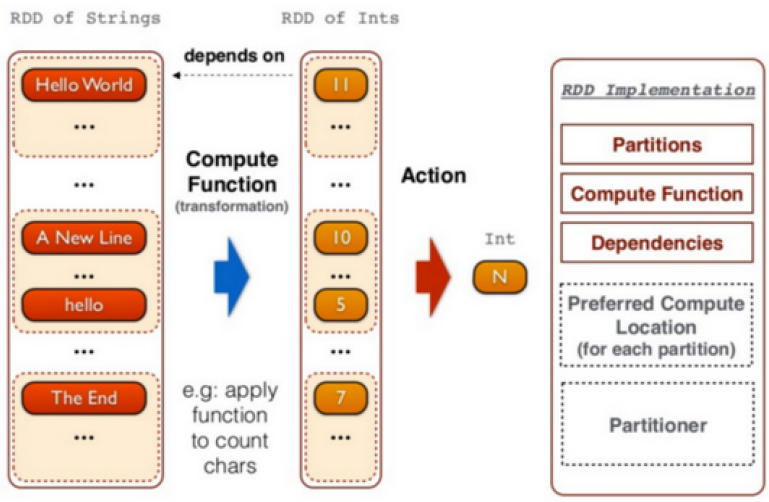


Schéma des RDD, transformations et actions. Source : Lilia Sfaxi, INSAT Tunisie, « TP Big Data », TP2 (https://insatunisia.github.io/TP-BigData/tp2/).

2.2 MODULES PRINCIPAUX

Spark Core	Moteur d'exécution, gestion des RDDs et de la mémoire distribuée
Spark SQL	Traitement de données structurées via DataFrames et Datasets
Spark Streaming	Traitement de flux en temps réel (micro-batches)
MLlib	Bibliothèque d'apprentissage automatique distribué
GraphX	Traitement de graphes à grande échelle

2.3 INTÉGRATION AVEC HADOOP YARN

Dans ce TP, Spark s'exécute sur Hadoop YARN, qui prend en charge la gestion des ressources pour le déclenchement et l'exécution des jobs Spark. L'environnement est composé de trois contenaires Docker : hadoop-master (NameNode + ResourceManager), hadoop-worker1 et hadoop-worker2 (DataNodes + NodeManagers).

3. INSTALLATION ET DÉMARRAGE DU CLUSTER

3.1 LANCEMENT DES CONTENAIRES DOCKER

Les contenaires Docker utilisés dans ce TP sont les mêmes que ceux du TP1, avec Spark pré-installé. Le démarrage se fait avec la commande suivante :

```
root@ubuntu-24-VMware-Virtual-Platform:~# docker start hadoop-master
hadoop-worker1 hadoop-worker2
```

Puis on entre dans le nœud maître et on lance les démons HDFS et YARN :

```
root@ubuntu-24-VMware-Virtual-Platform:~# docker exec -it hadoop-master bash
root@hadoop-master:~# ./start-hadoop.sh
```

3.2 VÉRIFICATION DES DÉMONS

La commande jps permet de vérifier que tous les démons sont bien démarrés. Sur le nœud master, on devrait obtenir :

```
root@hadoop-master:~# jps
176 NameNode
419 SecondaryNameNode
979 Jps
660 ResourceManager
root@hadoop-master:~#
```

Sur chaque nœud worker :

```
root@hadoop-worker1:~# jps
203 NodeManager
76 DataNode
381 Jps
```

4. TEST DE SPARK AVEC SPARK-SHELL (SCALA)

4.1 PRÉPARATION DU FICHIER DE TEST

Avant de lancer Spark-Shell, on crée un fichier texte de test ***file1.txt*** sur le nœud master, puis on le charge dans HDFS :

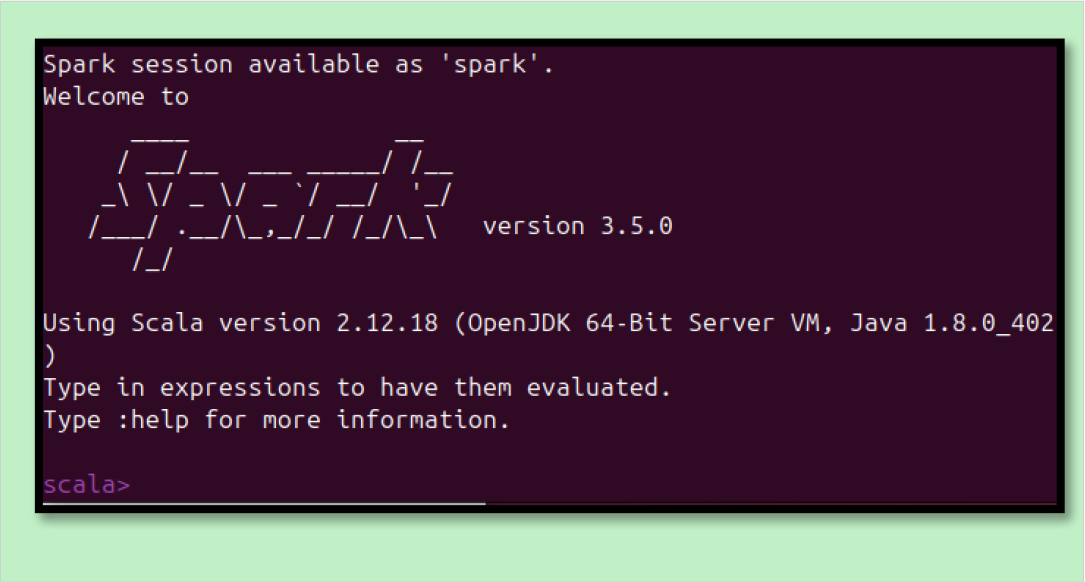
```
root@hadoop-master:~# cat file1.txt
Hello Spark Wordcount!
Hello Hadoop Also :)
root@hadoop-master:~# hdfs dfs -put file1.txt
```

4.2 LANCEMENT DE SPARK-SHELL

On lance Spark-Shell avec la commande suivante :

```
root@hadoop-master:~# spark-shell
```

Au lancement, Spark-Shell affiche la version de Spark ainsi que le contexte Scala disponible (sc pour SparkContext, spark pour SparkSession).



4.3 WORDCOUNT EN SCALA

On exécute le code WordCount suivant, ligne par ligne dans le shell :

```scala
// Étape 1 — Charger le fichier depuis HDFS
val lines = sc.textFile("file1.txt")

// Étape 2 — Séparer les mots
val words = lines.flatMap(_.split("\\s+"))

// Étape 3 — Mapper chaque mot avec la valeur 1
val wc = words.map(w => (w, 1)).reduceByKey(_ + _)

// Étape 4 — Sauvegarder le résultat dans HDFS
wc.saveAsTextFile("file1.count")
```

4.4 EXPLICATION DES TRANSFORMATIONS ET ACTION

Chaque opération correspond à une étape du pipeline de traitement distribué :

Opération	Type	Description

`textFile()`	Création de RDD	Charge le fichier depuis HDFS en RDD de lignes
`flatMap()`	Transformation	Découpe chaque ligne en liste de mots (lazy)
`map(w => (w,1))`	Transformation	Crée un Pair RDD (mot, 1) pour chaque mot
`reduceByKey()`	Transformation	Additionne les 1 pour chaque clé (mot)
`saveAsTextFile()`	Action	Déclenche l'exécution et sauvegarde le résultat

4.5 RÉSULTATS OBTENUS

Après exécution, on télécharge le répertoire de sortie depuis HDFS et on consulte les fichiers résultats :

```
root@hadoop-master:~# hdfs dfs -get file1.count
```

Le contenu des fichiers *part-00000* et *part-00001* affiche les paires (mot, fréquence) :

```
root@hadoop-master:~# cd file1.count/
root@hadoop-master:~/file1.count# cat part-00000
(Hello,2)
(Wordcount!,1)
root@hadoop-master:~/file1.count# cat part-00001
(Spark,1)
(:),1)
(Also,1)
(Hadoop,1)
```

4.6 EXEMPLE : RELEVER LES MOTS LES PLUS FRÉQUENTS DANS UN FICHIER

- Etape 1 – Créer un RDD à partir d'un fichier texte de hadoop

```
scala> val docs = sc.textFile("file1.txt")
docs: org.apache.spark.rdd.RDD[String] = file1.txt MapPartitionsRDD[1] at textFile at <console>:23
```

- Etape 2 – Convertir les lignes en minuscule

```
scala> val lower = docs.map(line => line.toLowerCase)
lower: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at map at <console>:23
```

- Etape 3 – Séparer les lignes en mots

```
scala> val words = lower.flatMap(line => line.split("\\s+"))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[3] at flatMap at <console>:23
```

- Etapes 4 – Produire les tuples (mot,1)

```
scala> val counts = words.map(word => (word,1))
counts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[4] at map at <console>:23
```

- Etape 5 – Compter tous les mots

```
scala> val freq = counts.reduceByKey(_ + _)
freq: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[5] at reduceByKey at <console>:23
```

- Etape 6 – Inverser les tuples (action de sélection des 3 premiers)

```
scala> val top = freq.map(_.swap).top(3)
[Stage 0:>
       top: Array[(Int, String)] = Array((2,hello), (1,wordcount!), (1,spark))
```

4.6.1 EXPLICATION DES TRANSFORMATIONS ET ACTION

Opération	Type	RDD resultant
textFile()	Création	`RDD[String]` (lignes)
map(toLowerCase)	Transformation	`RDD[String]` (minuscules)
flatMap(split)	Transformation	`RDD[String]` (mots)

map((mot,1))	Transformation	`RDD[(String, Int)]`
reduceByKey(+)	Transformation	`RDD[(String, Int)]`
map(swap).top(3)	Action	`Array[(Int, String)]`

5. SPARK BATCH EN JAVA (WORDCOUNT)

5.1 CRÉATION DU PROJET MAVEN

On crée un projet Maven dans VSCode avec la configuration suivante :

- **Groupe Id**: ***spark.batch***
- **Artifact Id : *wordcount***

5.2 DÉPENDANCES POM.XML

Les dépendances Spark Core et la configuration du compilateur Java 1.8 sont ajoutées au fichier ***pom.xml*** :

```xml
<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</properties>
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-core_2.13</artifactId>
  <version>3.5.0</version>
</dependency>
```

5.3 CODE WORDCOUNTTASK.JAVA

La classe WordCountTask implémente le WordCount en Java. Les étapes clés sont l'initialisation du JavaSparkContext, le chargement du fichier, la chaîne de transformations et la sauvegarde du résultat.

```java
package spark.batch.tp21;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import scala.Tuple2;

import java.util.Arrays;
import com.google.common.base.Preconditions;
```

```java
public class WordCountTask {
    private static final Logger LOGGER = LoggerFactory.getLogger(clazz: WordCountTask.class);

    Run | Debug
    public static void main(String[] args) {
        Preconditions.checkArgument(args.length > 1, "Please provide the path of input file
        new WordCountTask().run(args[0], args[1]);
    }

    public void run(String inputFilePath, String outputDir) {
        String master = "local[*]";
        SparkConf conf = new SparkConf()
                .setAppName(WordCountTask.class.getName())
                .setMaster(master);
        JavaSparkContext sc = new JavaSparkContext(conf);

        JavaRDD<String> textFile = sc.textFile(inputFilePath);
        JavaPairRDD<String, Integer> counts = textFile
                .flatMap(s -> Arrays.asList(s.split(" ")).iterator())
                .mapToPair(word -> new Tuple2<>(word, 1))
                .reduceByKey((a, b) -> a + b);
        counts.saveAsTextFile(outputDir);
        sc.close();
    }
}
```

5.4 TEST EN LOCAL

Pour tester le code en local, on insère un fichier texte (***loremipsum.txt***) dans src/main/resources et on configure les arguments de lancement :

- **Arg1** : ***wordcount-spark/src/main/resources/input/loremipsum.txt***
- **Arg2** : ***wordcount-spark/src/main/resources/output***

wordcount-spark > src > main > resources > output > part-00000

```
(laboramus,,1)
(nihil,1)
(putent,1)
(utinam,2)
(ea,,1)
(scripta,2)
(deseruisse,1)
(ea,4)
(constituto,1)
(tota,1)
(Albucius,1)
(iracundia,1)
(nam,2)
(evertitur,1)
(signiferumque,1)
(iusto,1)
(nec,,1)
(nostrum,1)
(aeterno,1)
(Ea,1)
(has.,1)
(consequuntur,1)
(esse,1)
```

wordcount-spark > src > main > resources > output > part-00001

```
(in,,2)
(accusamus,1)
(offendit,,1)
(Alii,1)
(delenit,1)
(ocurreret,1)
(Eam,1)
(qui.,1)
(maluisset,1)
(vel,,2)
(id,,1)
(Lorem,1)
(dicat,1)
(molestiae.,1)
(amet,,1)
(meliore,1)
(adolescens,,1)
(graeci,1)
(eum,1)
(usu.,1)
(dicant,1)
(qui,2)
(vituperata,1)
```

5.5 DÉPLOIEMENT SUR LE CLUSTER

On package l'application en JAR, on le copie dans le contenaire master, puis on le soumet avec spark-submit :

```
# Package Maven
mvn package

# Copie du JAR dans le contenaire
docker cp target/wordcount-spark-1.0-SNAPSHOT.jar hadoop-master:/root/wordcount-
spark.jar
# Exécution en mode local
spark-submit --class spark.batch.tp21.WordCountTask \
  --master local \
  wordcount-spark.jar input/purchases.txt out-spark

# Exécution en mode YARN cluster
spark-submit --class spark.batch.tp21.WordCountTask \
  --master yarn --deploy-mode cluster \
  wordcount-spark.jar input/purchases.txt out-spark2
```

5.6 MODES DE DÉPLOIEMENT YARN

Mode cluster	Le Spark Driver s'exécute dans le cluster (YARN Application Master). Recommandé pour la production.
Mode client	Le Spark Driver s'exécute sur la machine cliente. Si la machine s'éteint, le job s'arrête. Recommandé pour les jobs interactifs.

5.7 RÉSULTATS

Le répertoire out-spark créé dans HDFS contient les fichiers *part-00000* et *part-00001* avec le comptage des mots du fichier *purchases.txt*.

```
root@hadoop-master:~# hdfs dfs -tail out-spark/part-00000
.2,87)
(375.38,73)
(291.13,65)
(446.42,77)
(452.49,81)
(253.31,87)
(178.68,66)
(172.95,89)
(424.55,93)
(105.64,94)
(114.97,68)
(Arlington,40348)
(24.97,79)
(195.09,90)
(22.02,73)
(469.7,88)
(186.23,83)
(384.3,81)
(64.6,90)
(449.9,83)
(466.39,63)
(2012-04-10,11215)
(408.73,68)
(138.75,85)
```

```
root@hadoop-master:~# hdfs dfs -tail out-spark/part-00001
7)
(138.69,89)
(257.3,75)
(481.51,77)
(427.86,72)
(478.53,80)
(319.84,88)
(236.4,87)
(56.64,74)
(171.44,79)
(345.27,83)
(496.97,86)
(266.09,89)
(303.98,93)
(33.94,71)
(333.86,67)
(65.95,75)
(268.1,94)
(131.11,74)
(118.76,111)
(458.22,83)
(113.4,77)
(398.76,80)
(61.02,74)
(348.08,77)
(100.93,82)
```

5.7.1 RÉSULTAT EN MODE YARN CLUSTER

Après exécution en mode YARN cluster, le répertoire *out-spark2* est créé dans HDFS. On peut vérifier son contenu avec:

```
hdfs dfs -ls out-spark2
hdfs dfs -tail out-spark2/part-00000
```

```
root@hadoop-master:~#   spark-submit  --class spark.batch.tp21.WordCountTask
 --master yarn --deploy-mode cluster wordcount-spark.jar input/purchases.txt
 out-spark2
```

```
root@hadoop-master:~# hdfs dfs -ls out-spark2
Found 3 items
-rw-r--r--   2 root supergroup          0 2026-03-22 01:08 out-spark2/_SUCCESS
-rw-r--r--   2 root supergroup     300531 2026-03-22 01:08 out-spark2/part-00000
-rw-r--r--   2 root supergroup     300511 2026-03-22 01:08 out-spark2/part-00001
root@hadoop-master:~#
```

```
root@hadoop-master:~# hdfs dfs -tail out-spark2/part-00000
.2,87)
(375.38,73)
(291.13,65)
(446.42,77)
(452.49,81)
(253.31,87)
(178.68,66)
(172.95,89)
(424.55,93)
(105.64,94)
(114.97,68)
(Arlington,40348)
(24.97,79)
(195.09,90)
(22.02,73)
(469.7,88)
(186.23,83)
(384.3,81)
(64.6,90)
(449.9,83)
(466.39,63)
(408.73,68)
(2012-04-10,11215)
(138.75,85)
(28.66,89)
```

```
root@hadoop-master:~# hdfs dfs -tail out-spark2/part-00001
7)
(138.69,89)
(257.3,75)
(481.51,77)
(427.86,72)
(478.53,80)
(319.84,88)
(236.4,87)
(56.64,74)
(171.44,79)
(345.27,83)
(496.97,86)
(266.09,89)
(303.98,93)
(33.94,71)
(333.86,67)
(65.95,75)
(268.1,94)
(131.11,74)
(118.76,111)
(458.22,83)
(113.4,77)
(398.76,80)
(61.02,74)
(348.08,77)
(100.93,82)
(261.02,82)
```

6. SPARK STREAMING (STRUCTURED STREAMING)

6.1 PRÉSENTATION

Spark Streaming permet le traitement de données en temps réel. Les données sont reçues depuis diverses sources (Kafka, sockets TCP, Kinesis...) et divisées en micro-batches traités par le moteur Spark. Dans ce TP, on utilise Structured Streaming, l'API moderne de Spark basée sur Spark SQL et les DataFrames.

6.2 CONFIGURATION DU PROJET MAVEN

Un nouveau projet Maven est créé avec les dépendances spark-core, spark-streaming et spark-sql :

```
<groupId>spark.streaming</groupId>
<artifactId>stream</artifactId>

<!-- Dépendances principales -->
spark-core_2.13 : 3.5.0
spark-streaming_2.13 : 3.5.0
spark-sql_2.13 : 3.5.0 (scope: provided)
```

6.3 CODE DE LA CLASSE STREAM.JAVA

L'application Stream.java écoute les données sur le port 9999 en localhost, divise les lignes en mots, et affiche le comptage mis à jour toutes les secondes dans la console :

```
// Lecture du stream depuis le socket
Dataset<String> lines = spark.readStream()
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load()
    .as(Encoders.STRING());

// Découpage en mots et comptage
Dataset<Row> wordCounts = lines
    .flatMap(...)
    .groupBy("value").count();

// Lancement de la query avec trigger d'1 seconde
wordCounts.writeStream()
    .outputMode("complete")
    .format("console")
    .trigger(Trigger.ProcessingTime("1 second"))
    .start();
```

6.4 DÉPLOIEMENT ET TEST SUR LE CLUSTER

On génère le JAR, on le copie sur le master sous le nom stream-1.jar, puis on installe netcat et on ouvre deux terminaux :

```
# Terminal 1 — Source de streaming (netcat)
apt update && apt install netcat
nc -lk 9999

# Terminal 2 — Lancement de l'application Spark
spark-submit --class spark.streaming.tp22.Stream \
  --master local stream-1.jar > out
```

On saisit du texte dans le terminal netcat (Terminal 1). Spark (Structured Streaming) le reçoit, le traite par micro-batch toutes les secondes, et affiche le résultat cumulé.

> **Trigger Interval :** *Le trigger par défaut est ProcessingTime(0), ce qui signifie que Spark traite les données aussi rapidement que possible. Ici on utilise ProcessingTime('1 second') pour un traitement toutes les secondes.*

6.5 RÉSULTATS

Le fichier out contient les micro-batches successifs avec le comptage des mots reçus. On observe que le comptage est cumulatif (outputMode complete) — chaque batch affiche tous les mots depuis le début du stream.

```
root@hadoop-master:~# cat out
-------------------------------------------
Batch: 0
-------------------------------------------
+-----+-----+
|value|count|
+-----+-----+
+-----+-----+

-------------------------------------------
Batch: 1
-------------------------------------------
+-----+-----+
|value|count|
+-----+-----+
|     |    1|
+-----+-----+

-------------------------------------------
Batch: 2
-------------------------------------------
+------+-----+
| value|count|
+------+-----+
|Rachid|    1|
|Franck|    1|
|    et|    1|
|      |    1|
|Abdoul|    1|
+------+-----+
```

7. CONCLUSION

Ce TP a permis de mettre en pratique les deux modes de traitement fondamentaux d'Apache Spark dans un environnement distribué réel.

Le traitement batch avec Spark-Shell et Java démontre la puissance du modèle RDD et la facilité d'écriture de pipelines de données distribués grâce aux opérations fonctionnelles (map, flatMap, reduceByKey). Le déploiement via spark-submit sur YARN cluster valide l'intégration complète avec l'écosystème Hadoop.

Le traitement streaming avec Structured Streaming illustre le paradigme moderne du traitement en temps réel par micro-batches. L'API DataFrames offre une abstraction de haut niveau qui simplifie considérablement le code par rapport à l'ancienne API DStream.

Notions maîtrisées	RDDs, transformations lazy, actions, Pair RDDs, spark-submit
Modes de déploiement	Local, YARN client, YARN cluster
APIs utilisées	JavaSparkContext (batch), SparkSession + Structured Streaming
Outils	Spark-Shell (Scala), Maven, Docker, netcat